

**САНКТ-ПЕТЕРБУРГСКИЙ НАЦИОНАЛЬНЫЙ
ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО**

Дисциплина: Бэк-энд разработка

Отчет

Лабораторная работа №3

«Контейнеризация написанного приложения средствами docker»

Выполнил:

Аплеев Дмитрий

К3339

Проверил:

Добряков Д. И.

Санкт-Петербург

2026 г.

Задача

Написать Dockerfile для каждого сервиса, общий docker-compose.yml и настроить сеть между сервисами.

Вариант 5. Сервис для обмена рецептами и кулинарных блогов.

Ход работы

1. Dockerfile

У всех четырёх сервисов одинаковый Dockerfile.

```
FROM node:20
WORKDIR /app

COPY package.json package-lock.json ./
RUN npm ci

COPY tsconfig.json ./
COPY src ./src
RUN npm run build

CMD ["node", "dist/app.js"]
```

Строка	Что делает
FROM node:20	берёт готовый образ с Node 20
WORKDIR /app	рабочая папка внутри образа
COPY package.json, RUN npm ci	ставит зависимости, при изменении кода этот шаг берётся из кеша
COPY src, RUN npm run build	копирует код и собирает TypeScript в папку dist
CMD	команда при запуске контейнера

Таблица 1. Dockerfile

Файл .dockerignore не пускает в образ node_modules, dist и .env. Зависимости ставятся внутри образа под Linux, а пароли в образ не попадают.

2. Контейнеры

В docker-compose.yml восемь контейнеров.

Контейнер	Образ	Роль
user-db, recipe-db, social-db	postgres:16	три отдельные базы, данные хранятся в томах
rabbitmq	rabbitmq:3-management	очередь сообщений

Контейнер	Образ	Роль
user-service, recipe-service, social-service	свой Dockerfile	сервисы
bff-service	свой Dockerfile	единая точка входа, единственный открыт наружу

Таблица 2. Контейнеры

Пример базы и сервиса.

```

recipe-db:
  image: postgres:16
  environment:
    POSTGRES_USER: postgres
    POSTGRES_PASSWORD: ${DB_PASSWORD}
    POSTGRES_DB: recipe_db
  volumes:
    - recipe-db-data:/var/lib/postgresql/data
  healthcheck:
    test: ['CMD-SHELL', 'pg_isready -U postgres']

recipe-service:
  build: ./recipe-service
  environment:
    APP_HOST: 0.0.0.0
    APP_PORT: 3002
    DB_HOST: recipe-db
    DB_ENTITIES: dist/models/*.entity.js
    SOCIAL_SERVICE_URL: http://social-service:3003/api
    RABBITMQ_URL: amqp://guest:guest@rabbitmq:5672
    ...
  depends_on:
    recipe-db:
      condition: service_healthy
    rabbitmq:
      condition: service_healthy

```

3. Сеть

Compose создаёт общую сеть, в которой контейнеры находят друг друга по имени сервиса. Поэтому в адресах стоят `recipe-db`, `rabbitmq`, `social-service` вместо `localhost`. Сервисы слушают `0.0.0.0`, иначе соседи до них не достучатся. Наружу открыт только BFF на `127.0.0.1:3004` и панель RabbitMQ на `127.0.0.1:15672`. Базы и остальные сервисы доступны только внутри сети.

4. Порядок запуска и секреты

Сначала стартуют базы и RabbitMQ. Сервисы ждут, пока их `healthcheck` станет `healthy`, и только потом запускаются. BFF стартует после сервисов. Пароль базы, ключ JWT и служебный ключ лежат в файле `.env`

рядом с compose и подставляются через `${...}`. Файл `.env` добавлен в `.gitignore`.

Для отладки есть `docker-compose.override.yml`. Он открывает порты 3001, 3002 и 3003, чтобы смотреть Swagger сервисов. На сервере он не используется.

5. Запуск

```
docker compose up -d --build
docker compose ps
```

NAME	STATUS	PORTS
lab2-bff-service-1	Up	127.0.0.1:3004->3004/tcp
lab2-rabbitmq-1	Up (healthy)	127.0.0.1:15672->15672/tcp
lab2-recipe-db-1	Up (healthy)	5432/tcp
lab2-recipe-service-1	Up	
lab2-social-db-1	Up (healthy)	5432/tcp
lab2-social-service-1	Up	
lab2-user-db-1	Up (healthy)	5432/tcp
lab2-user-service-1	Up	

Приложение отвечает по адресу `http://localhost:3004/api/v1`. Порты сервисов снаружи закрыты.

Вывод

Приложение упаковано в Docker. Весь проект из восьми контейнеров поднимается одной командой, сервисы общаются по именам во внутренней сети, наружу открыт только BFF.